Autor:
Ana Isabel Vegas

ibertech

end2end dedication

CURSO

Desarrollo Web con REACT

www.ibertech.es

ibertech

end2end dedication

Tema 1 Introducción a ReactJS

Tema 2 Componentes y estado

Tema 3 JSX

Tema 4 Eventos

Tema 5 Formularios

Tema 6 ReactJS y datos

Tema 7 ReactJS y servidor

ibertech

end2end dedication

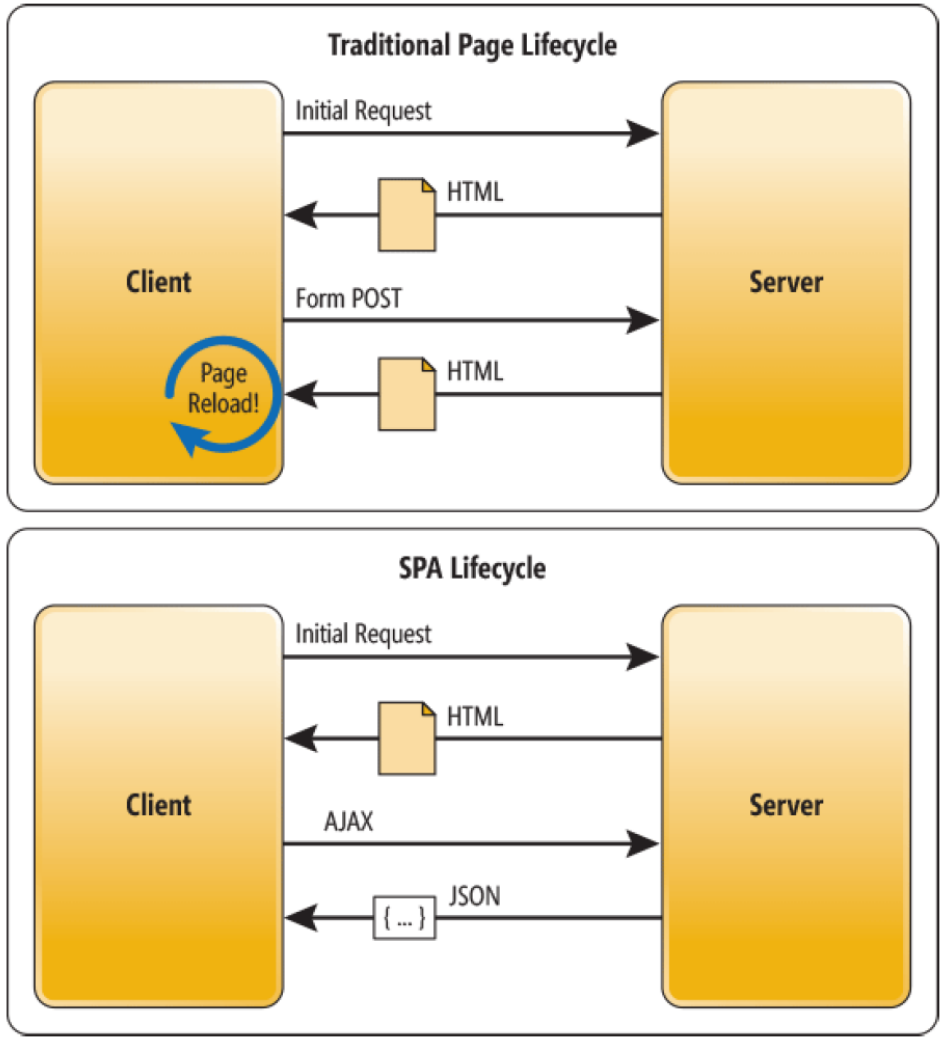
Tema 1

Introducción a ReactJS

Que es React?

- ReactJS es una librería Javascript de código abierto enfocada en la visualización para crear interfaces de usuario.
- React fue creada por Jordan Walke, un ingeniero de software en Facebook para solucionar los problemas internos con XHP, un marco de componentes de HTML para PHP. Fue lanzada en 2013 y es mantenida por Facebook, Instagram y una comunidad de desarrolladores.
- Nace para resolver los problemas de escalabilidad en aplicaciones con una gran flujo de datos y permite crear componentes de interfaz de usuario reutilizables.
- Ofrece grandes beneficios en rendimiento y modularidad, que promueve un flujo muy claro de datos y eventos, facilitando la planificación y desarrollo de páginas web complejas.
- Sitio Oficial: https://es.react.dev/

SPA


Traditional Page Lifecycle
Client
Server
Initial Request
HTML
Form POST
Page Reload!
HTML
SPA Lifecycle
Client
Server
Initial Request
HTML
AJAX
JSON
{ ... }

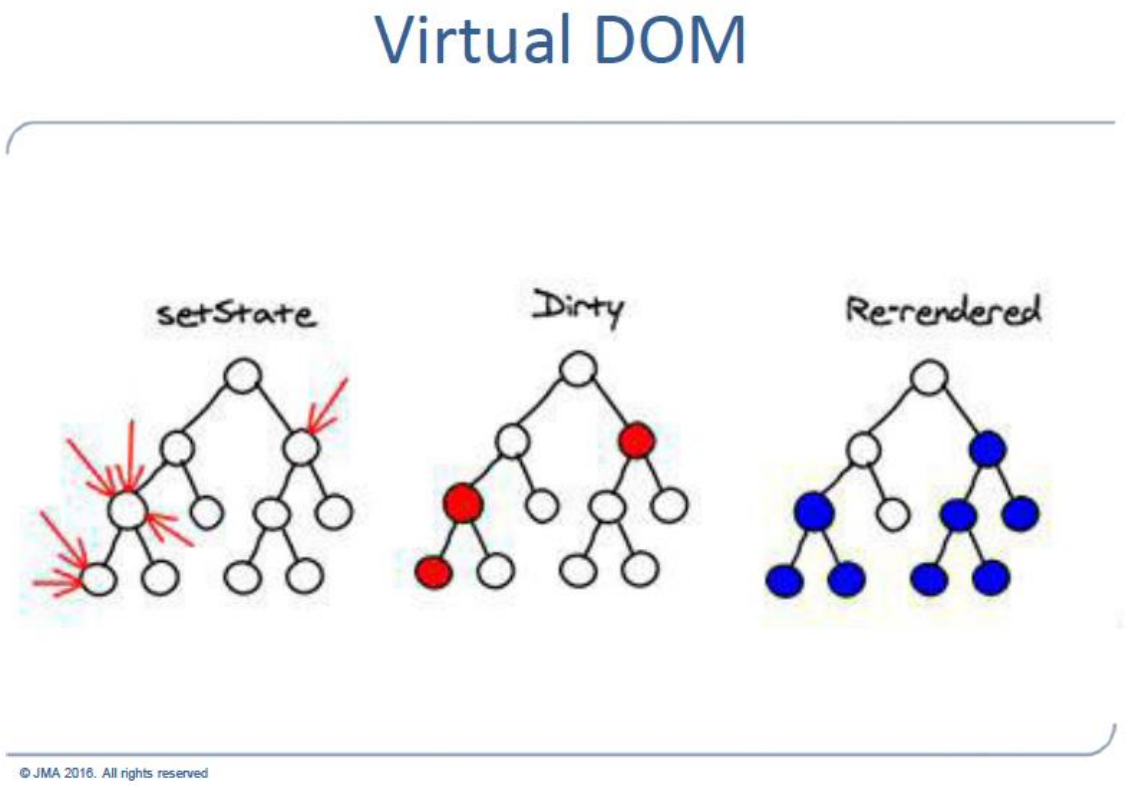
Virtual DOM

- El secreto de ReactJS para tener un rendimiento muy alto, es que implementa algo llamado Virtual DOM y en vez de renderizar todo el DOM en cada cambio, que es lo que normalmente se hace, este hace los cambios en una copia en memoria y después usa un algoritmo para comparar las propiedades de la copia en memoria con las de la versión del DOM y así aplicar cambios exclusivamente en las partes que varían.

- El algoritmo diferencial se encarga de actualizar el nuevo DOM comparándolo con el previo.

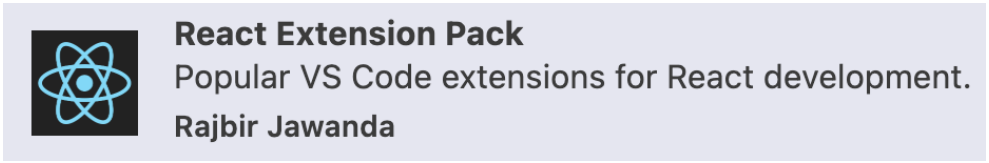
- En lugar de generar directamente elementos HTML y modificar el DOM, que es una operación muy lenta, los componentes de ReactJS generan (renderizan) un modelo del DOM en memoria. Una vez generado el Virtual DOM completo, ReactJS se encarga de buscar las diferencias entre el DOM real y el virtual y realizar únicamente las modificaciones necesarias sobre el DOM real.

Virtual DOM


Virtual DOM
setState
Dirty
Re-rendered
© JMA 2016. All rights reserved

Que necesito tener instalado?

- Node.js para poder utilizar el comando npm
- Visual Studio Code como entorno de desarrollo
- React Extension Pack en VSC



Node.js

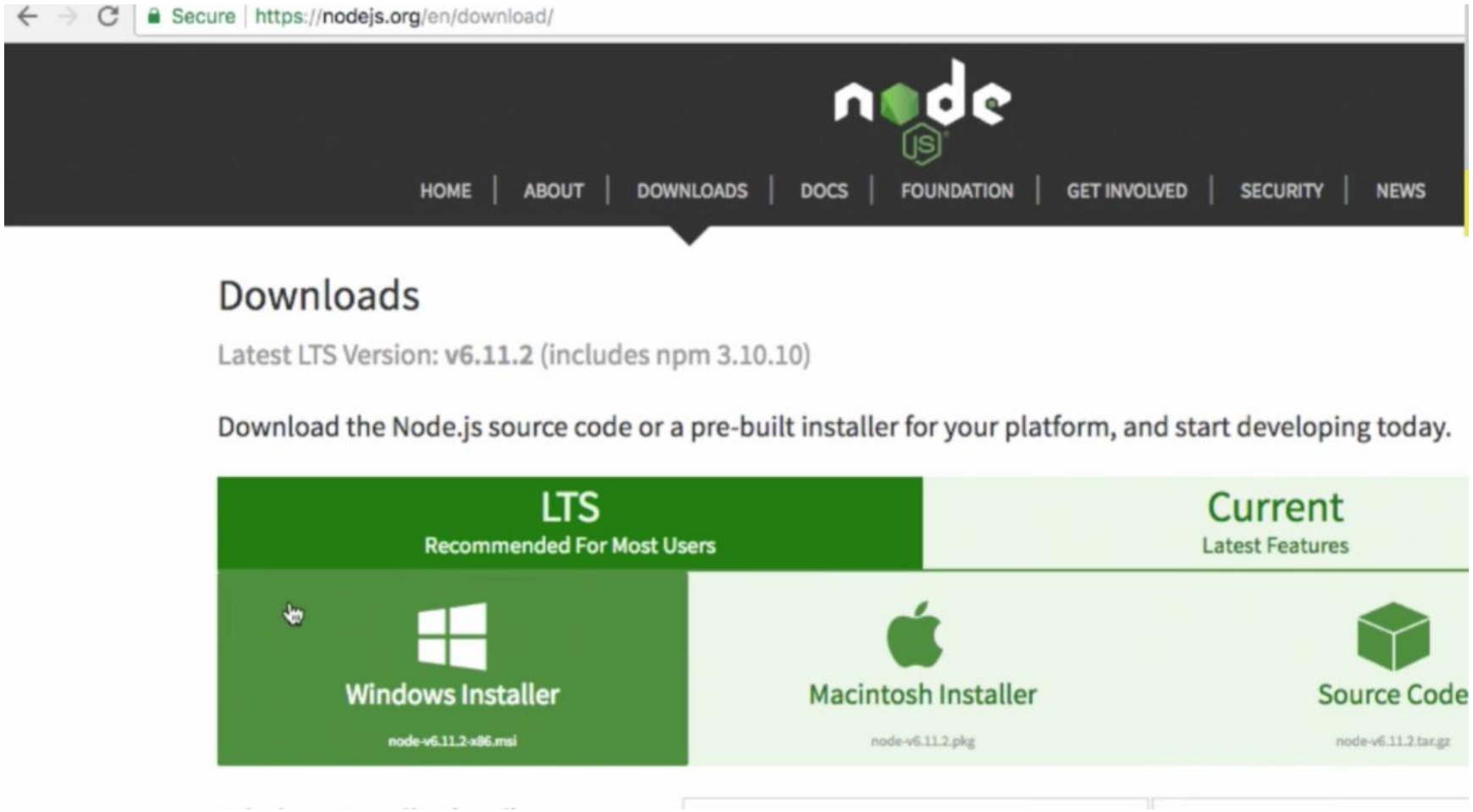
- Node.js, en la consola validamos que tenemos instalado npm con el comando:

```
npm -v
```

- De no ser asi nos lo descargamos de la pagina oficial:

https://nodejs.org/en/download/prebuilt-installer

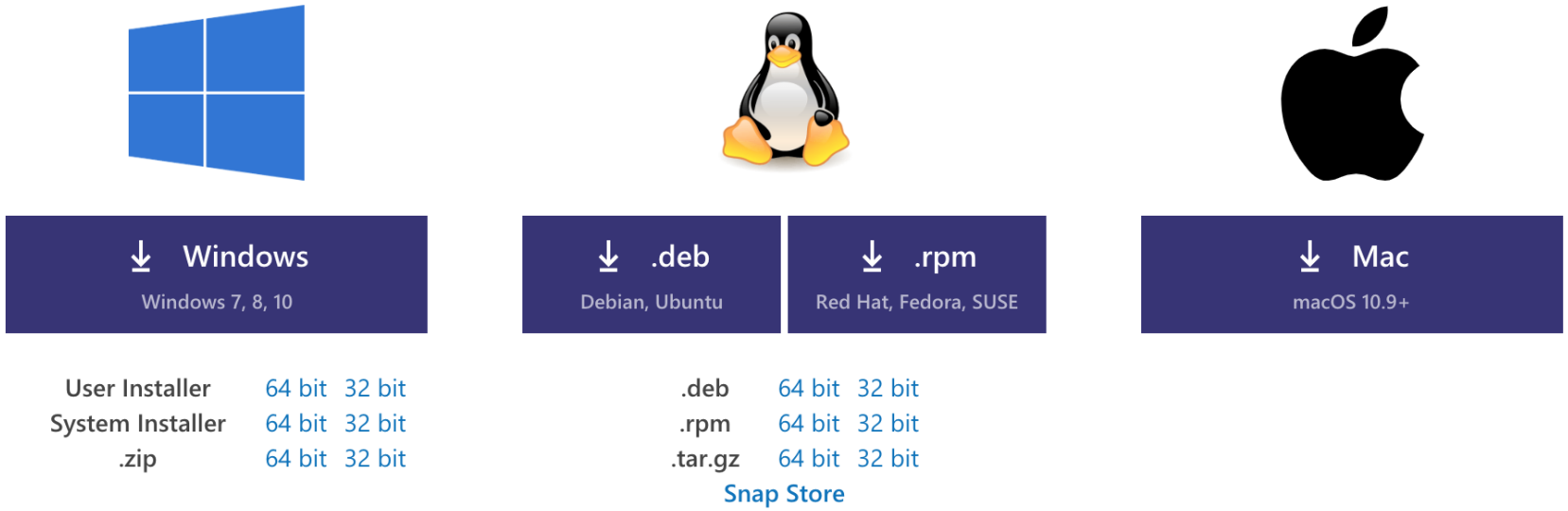
Node.js


Secure | https://nodejs.org/en/download/
node
HOME | ABOUT | DOWNLOADS | DOCS | FOUNDATION | GET INVOLVED | SECURITY | NEWS
Downloads
Latest LTS Version: v6.11.2 (includes npm 3.10.10)
Download the Node.js source code or a pre-built installer for your platform, and start developing today.
LTS
Recommended For Most Users
Current
Latest Features
Windows Installer
node-v6.11.2-x86.msi
Macintosh Installer
node-v6.11.2.pkg
Source Code
node-v6.11.2.tar.gz

Visual Studio Code

- Se puede descargar de esta url:

  https://code.visualstudio.com/download

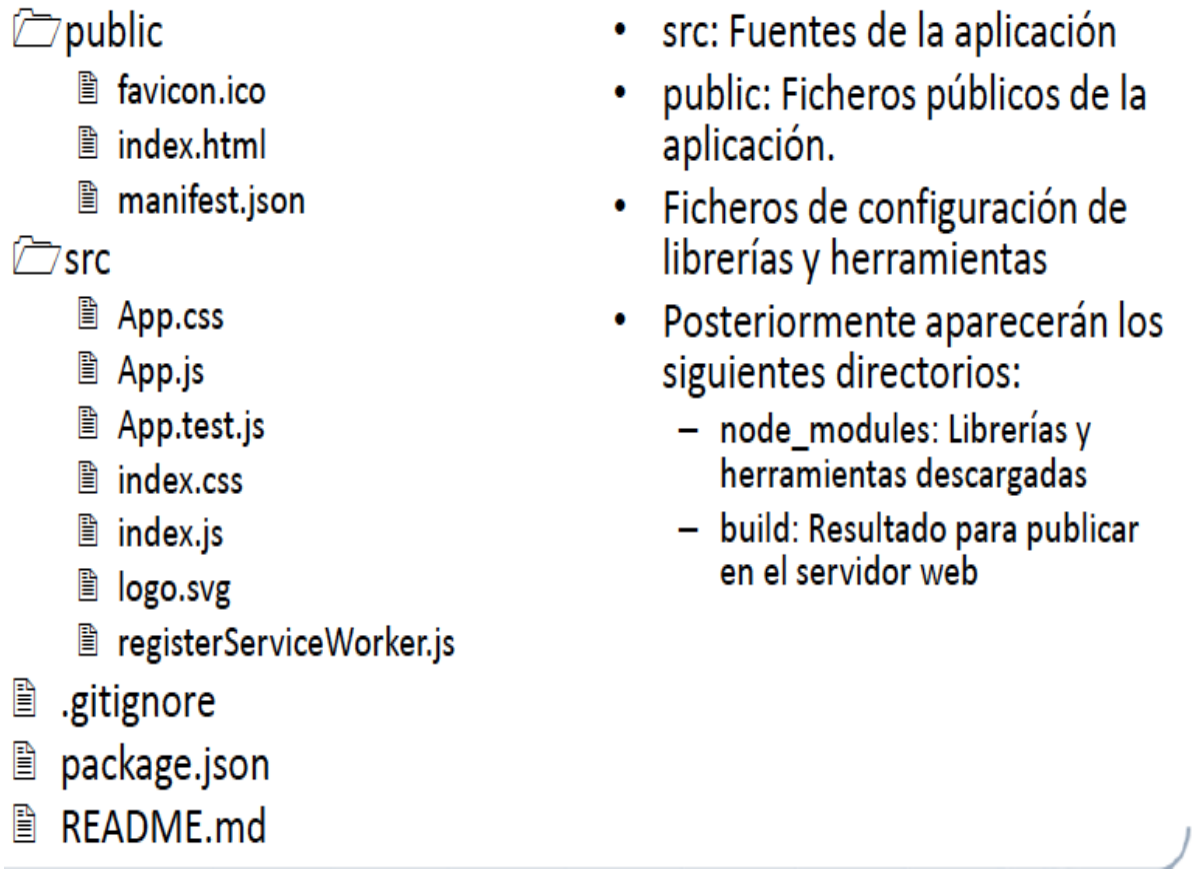


Como puedo crear un proyecto REACT?

- Primero, asegúrate de tener Node instalado.
- Utilizaremos Vite que es una herramienta que nos facilita el crear el proyecto, instalar dependencias y arrancarlo.

```
npm create vite@latest
```

Estructura de un proyecto



- src: Fuentes de la aplicación
- public: Ficheros públicos de la aplicación.
- Ficheros de configuración de librerías y herramientas
- Posteriormente aparecerán los siguientes directorios:
  - node_modules: Librerías y herramientas descargadas
  - build: Resultado para publicar en el servidor web

- En las ultimas versiones index.html no esta ubicado e la carpeta public sino en la raíz del proyecto

Pruebas y despliegue

- $ npm test
  - Inicia el entorno de pruebas en modo interactivo.
- $ npm run build
  - Genera la versión para producción en la carpeta /build, agrupando y minimizando para obtener un mejor rendimiento. Incluyen valores hash en los nombres de los archivo para impedir los problemas de las cache con el versionado.
  - De forma predeterminada, también incluye un service worker para cargar desde la memoria caché local en las futuras visitas.
- $ npm run eject
  - Elimina la herramienta y dependencias de compilación, después de copiar los archivos de configuración en el directorio de la aplicación.
  - Una vez hecho no tiene vuelta atrás.

ibertech

end2end dedication

Tema 2

Componentes y estado

Componentes

- Es una función de JavaScript que devuelve una interfaz de usuario.

- Componente Cabecera:

```
function Cabecera() {
  return (
    <header>
      <h1>Mi tienda</h1>
      <nav>
        <a href="/">Inicio</a>
        <a href="/productos">Productos</a>
      </nav>
    </header>
  );
}
```

Componente Producto

```
function Producto() {
  return (
    <article>
      <h2>Teclado</h2>
      <p>29,99 €</p>
      <button>Comprar</button>
    </article>
  );
}
```

- Como los utilizo en el componente principal:

```
function App() {
  return (
    <>
      <Cabecera />
      <Producto />
    </>
  );
}
```

Por qué utilizamos Componentes?

- Principalmente por:
  - Reutilización
  - Organización
  - Mantenimiento
  - Separación de responsabilidades
  - Composición

Formas de crear un componente

- Componente funcional:

```
function Saludo() {
  return <h1>Hola</h1>;
}
```

- Utilizando Arrow function:

```
const Saludo = () => {
  return <h1>Hola</h1>;
};
```

Props

- Las props permiten pasar información de un componente padre a un componente hijo.
- Podemos entenderlas como **parámetros de una función**.
- El padre puede enviar información al hijo, pero el hijo **no debería modificar** directamente las props que recibe.

```
function Saludo(props) {
  return <h1>Hola {props.nombre}</h1>;
}
```

```
function App() {
  return (
    <>
      <Saludo nombre="Ana" />
      <Saludo nombre="Carlos" />
      <Saludo nombre="Laura" />
    </>
  );
}
```



```
Hola Ana
Hola Carlos
Hola Laura
```

Props con destructuring

- Es otra forma mas cómoda de acceder a las props.

```
function Usuario({ nombre, edad, activo }) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>Edad: {edad}</p>
      <p>Activo: {activo ? 'Sí' : 'No'}</p>
    </div>
  );
}
```

```
<Usuario
  nombre="Ana"
  edad={25}
  activo={true}
/>
```

Tipos de props

- String, entre comillas:

```
<User name="Ana" />
```

- Números, se deben utilizar llaves:

```
<User age={25} />
```

- Booleanos, en minúsculas y entre llaves:

```
<User isAdmin={true} />
```

- Otra forma que es equivalente a:

```
<User isAdmin />
```



```
<User isAdmin={true} />
```

Tipos de props

- Objetos, se pasa entre llaves y se recibe igual en la función:

```
const user = {
  name: "Ana",
  age: 25,
  city: "Madrid"
};

function App() {
  return <User user={user} />;
}
```

```
function User({ user }) {
  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.age} años</p>
      <p>{user.city}</p>
    </div>
  );
}
```

- Arrays, se declaran con corchetes y se pasan entre llaves:

```
const products = ["Ordenador", "Ratón", "Teclado"];

<ProductList products={products} />
```

```
function ProductList({ products }) {
  return (
    <ul>
      {products.map(product => (
        <li key={product}>{product}</li>
      ))}
    </ul>
  );
}
```

Props vs variables internas

- Las props vienen del padre; las variables locales pertenecen al componente.

```
function User({ name }) {
  const message = "Hola";

  return (
    <div>
      <h2>{name}</h2>
      <p>{message}</p>
    </div>
  );
}
```

State

- El state representa información que puede cambiar durante la vida del componente y **cuyo cambio debe provocar una actualización de la interfaz.** Para eso utilizamos **useState**.

```
import { useState } from 'react';

function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Contador: {contador}</p>

      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

- ✓ **contador** es la variable.
- ✓ **useState(0)** la variable contador se inicializa a 0.
- ✓ **setContador** es la función. Cada vez que se pulsa el botón la función suma 1 a la variable contador y este cambio provoca que se actualice la interface

State

- Regla importante*: " No modificar directamente es estado "*
- **Nunca modifiques directamente una variable de estado. Utiliza siempre su setter.**

Incorrecto ✗

```
count = count + 1;
```

Correcto ✓

```
setCount(count + 1);
```

State de diferentes tipos

- String:

```
const [name, setName] = useState("");
```

```
setName("Carlos");
```

- Boolean:

```
const [isOpen, setIsOpen] = useState(false);
```

```
setIsOpen(true);
```

```
setIsOpen(!isOpen);
```

```
setIsOpen(previous => !previous);
```

- Arrays:

```
const [items, setItems] = useState([]);
```

```
setItems([...items, "React"]);
```

- Objetos:

```
const [user, setUser] = useState({
  name: "",
  age: 0
});
```

```
setUser({
  ...user,
  name: "Ana"
});
```

useState en formularios

```
function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <form>
      <input
        type="email"
        value={email}
        onChange={event => setEmail(event.target.value)}
      />

      <p>Email: {email}</p>
    </form>
  );
}
```

Props vs state

Props	State
Información recibida	Información interna
La proporciona el componente padre	La gestiona el propio componente
Es de solo lectura para el hijo	Puede cambiar
Permite configurar componentes	Permite que sean interactivos
Flujo padre → hijo	Puede provocar un nuevo render

Renderizado condicional

- Si queremos mostrar una interface diferente dependiendo del estado.
- Podemos utilizar el **operador ternario**: `condicion ? valorSiTrue : valorSiFalse`

```
function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <div>
      {isLoggedIn ? (
        <h1>Bienvenido</h1>
      ) : (
        <h1>Inicia sesión</h1>
      )}

      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        Cambiar estado
      </button>
    </div>
  );
}
```

Renderizado condicional

- También podemos utilizar **&&**:

```
function UserPanel({ isLoggedIn }) {
  return (
    <div>
      <h1>Panel de usuario</h1>

      {isLoggedIn && (
        <button>Cerrar sesión</button>
      )}
    </div>
  );
}
```

Renderizado condicional

- Otra opción es utilizar **if**:

```
function UserPanel({ isLoggedIn }) {
  if (!isLoggedIn) {
    return <p>Debes iniciar sesión</p>;
  }

  return (
    <div>
      <h1>Panel de usuario</h1>
      <button>Cerrar sesión</button>
    </div>
  );
}
```

ibertech

end2end dedication

Tema 3

JSX

JSX

- JSX es una extensión de sintaxis que nos permite describir la interfaz de usuario utilizando una sintaxis similar a HTML dentro de JavaScript.
- JSX permite escribir una sintaxis parecida a HTML dentro de JavaScript.
- Ejemplos:

```
function App() {
  return <h1>Hola mundo</h1>;
}
```

```
const name = "Ana";

function App() {
  return <h1>Hola {name}</h1>;
}
```

JSX no es HTML

- Algunos atributos cambian porque JSX utiliza propiedades que siguen las convenciones de JavaScript.

HTML

```
<div class="container">
  <label for="name">Nombre</label>
</div>
```

JSX

```
<div className="container">
  <label htmlFor="name">Nombre</label>
</div>
```

HTML	JSX
class	className
for	htmlFor
tabindex	tabIndex
onclick	onClick
readonly	readOnly

JSX debe devolver un único elemento

- Esto no es válido:

```
function App() {
  return (
    <h1>Título</h1>
    <p>Descripción</p>
  );
}
```



- Necesitamos un elemento que englobe ambos:

```
function App() {
  return (
    <div>
      <h1>Título</h1>
      <p>Descripción</p>
    </div>
  );
}
```



JSX

- Las etiquetas deben cerrarse.
- En HTML algunas etiquetas pueden escribirse sin cierre explícito:

```
<img src="foto.jpg">
<input type="text">
```

- Pero en JSX deben cerrarse:

```
<img src="foto.jpg" />
<input type="text" />
```

- Ejemplos:

```
<br />
<hr />
<img src="foto.jpg" />
<input type="text" />
```

Expresiones JavaScript en JSX

- Para introducir JavaScript dentro de JSX utilizamos **{}**:

- Ejemplos:

```
const name = "Ana";

function App() {
  return <h1>Hola {name}</h1>;
}
```

```
<h1>{2 + 2}</h1>
```

```
<p>{name.toUpperCase()}</p>
```

- **Esto no funciona**, hay que utilizar operador ternario porque no es una expresión, es una instrucción.

```
<p>
  {if (isLoggedIn) {
    return "Usuario";
  }}
</p>
```



```
<p>
  {isLoggedIn ? "Usuario" : "Invitado"}
</p>
```

Atributos en JSX

- String directamente → comillas.

```
<img src="foto.jpg" alt="Foto de perfil" />
```

- JavaScript → llaves.

```
const imageUrl = "https://ejemplo.com/foto.jpg";

<img src={imageUrl} alt="Foto de perfil" />
```

Atributos booleanos

- Aquí tenemos distintas formas de utilizar atributos booleanos:

```
<button disabled={true}>
  Guardar
</button>
```

```
<button disabled>
  Guardar
</button>
```

```
const isDisabled = true;

<button disabled={isDisabled}>
  Guardar
</button>
```

- Con useState:

```
const [loading, setLoading] = useState(false);

<button disabled={loading}>
  Guardar
</button>
```

Estilos en JSX

- Esto no es correcto:

```
<div style="color: red">
```

- El estilo hay que pasarlo dentro de **{}**:

```
<div style={{ color: "red" }}>
  Hola
</div>
```

La primera {} significa voy a escribir código JavaScript

La segunda {} es el objeto que paso

```
<div style={{ color: "red", backgroundColor: "black" }}>
```

Propiedades CSS

- En CSS:

```
background-color
font-size
margin-top
```

- En JSX:

```
backgroundColor
fontSize
marginTop
```

- Ejemplo:

```
const styles = {
  color: "white",
  backgroundColor: "blue",
  fontSize: "20px",
  marginTop: "10px"
};

function App() {
  return <div style={styles}>Hola</div>;
}
```

CSS Externo

- Podemos utilizar CSS tradicional:

```css
.card {
  background-color: white;
  padding: 20px;
  border-radius: 8px;
}
```

- Lo insertamos en JSX:

```jsx
<div className="card">
  <h2>Producto</h2>
  <p>19,99 €</p>
</div>
```

Fragmentos

- Como hemos visto, JSX necesita un elemento raíz.
- Podemos utilizar un :

```
return (
  <div>
    <h1>Título</h1>
    <p>Descripción</p>
  </div>
);
```

- Pero también podemos utilizar un **Fragment**, que no genera ningún elemento adicional en el DOM:

Sintaxis corta

```
return (
  <>
    <h1>Título</h1>
    <p>Descripción</p>
  </>
);
```

es equivalente a esto:

```
import { Fragment } from "react";

return (
  <Fragment>
    <h1>Título</h1>
    <p>Descripción</p>
  </Fragment>
);
```

Listas

- Queremos convertir un array en elementos
- La key proporciona una identidad estable a cada elemento de una lista.
- key es una propiedad especial que React utiliza internamente, no es una prop.
- La key permite que React pueda identificar cada elemento de una lista. Por eso deberían ser:
  - únicas dentro de la lista
  - estables
  - asociadas al elemento

```
const products = [
  {
    id: 1,
    name: "Teclado",
    price: 50
  },
  {
    id: 2,
    name: "Ratón",
    price: 25
  },
  {
    id: 3,
    name: "Monitor",
    price: 200
  }
];
```

```
function ProductList() {
  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>
          {product.name} - {product.price} €
        </li>
      ))}
    </ul>
  );
}
```

Listas

- ¿Puedo utilizar el índice como key?
- Es habitual ver:

```
products.map((product, index) => (
  <li key={index}>
    {product.name}
  </li>
))
```

- Funciona técnicamente, pero no debería ser la opción por defecto.
- Si los elementos pueden: reordenarse, eliminarse, insertarse, cambiar de posición, … el índice puede provocar problemas de identidad.
- Es preferible:

```
key={product.id}
```

- **Si los datos tienen un ID estable, utiliza ese ID como key.**

ibertech

end2end dedication

Tema 4

Eventos

Eventos en REACT

- Un evento ocurre cuando el usuario hace algo y React nos permite ejecutar una función como respuesta a esa acción.
- React utiliza camelCase: onClick, no onclick.
- Pasamos una función, no una cadena de texto.

```
function App() {

  function handleClick() {
    console.log("Has hecho click");
  }

  return (
    <button onClick={handleClick}>
      Haz click
    </button>
  );
}
```

- Función inline:

```
<button onClick={() => console.log("Click")}>
  Pulsar
</button>
```

El objeto evento

- Cuando ocurre un evento, React nos proporciona información sobre él.
- Podemos recibirlo como parámetro:

```
function handleClick(event) {
  console.log(event.type);
}
```

```
<button onClick={handleClick}>
  Pulsar
</button>
```

- También podemos consultar el elemento que originó el evento:

```
function handleClick(event) {
  console.log(event.currentTarget);
}
```

Eventos sintéticos

- React nos proporciona un objeto de evento con información común para trabajar con las interacciones del usuario.
- El objeto que recibimos es un evento gestionado por React y proporciona una API consistente para trabajar con eventos.
- Podemos acceder a propiedades habituales:

```
function handleClick(event) {
  console.log("Tipo:", event.type);
  console.log("Elemento:", event.currentTarget);
}
```

- Para un campo de formulario:

```
function handleChange(event) {
  console.log(event.target.value);
}
```

target: Elemento que originó el evento.

currentTarget: Elemento al que está asociado el listener.

Pasar parámetros a una función

- Si la función recibe un argumento:

```
function deleteProduct(id) {
  console.log("Eliminar", id);
}
```

- No podemos hacer esto porque estamos ejecutando la función durante el renderizado:

```
<button onClick={deleteProduct(product.id)}>
```

- Esto es lo correcto:

```
<button onClick={() => deleteProduct(product.id)}>
  Eliminar
</button>
```

Evento onChange

- El valor del input está controlado por React. search es una variable de estado. A esto se le llama **Controlled component (componentes controlados).**

```
import { useState } from "react";

function Search() {
  const [search, setSearch] = useState("");

  function handleChange(event) {
    setSearch(event.target.value);
  }

  return (
    <div>
      <input
        type="text"
        value={search}
        onChange={handleChange}
      />

      <p>Buscando: {search}</p>
    </div>
  );
}
```

Evento onSubmit

- **preventDefault(); cancela** el comportamiento HTML por defecto y deja que React gestione lo que ocurre.

```
function LoginForm() {

  function handleSubmit(event) {
    event.preventDefault();

    console.log("Formulario enviado");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" />

      <button type="submit">
        Entrar
      </button>
    </form>
  );
}
```

Pasar funciones de padre a hijo mediante props

- Queremos que el padre decida qué ocurre cuando se pulsa el botón:
- Permite que un componente hijo pueda comunicar al padre que ha ocurrido algo.
- Desde el padre:

```
function App() {

  function handleClick() {
    console.log("Botón pulsado");
  }

  return (
    <Button onClick={handleClick} />
  );
}
```

Desde el hijo:

```
function Button({ onClick }) {
  return (
    <button onClick={onClick}>
      Pulsar
    </button>
  );
}
```

- Las props permiten pasar datos hacia abajo y las funciones permiten comunicar eventos hacia arriba.
- No es que React tenga un mecanismo especial de "subir datos"; el padre pasa una función y el hijo la ejecuta.

Paso de argumentos mediante funciones

```
function ProductList() {

  function handleDelete(id) {
    console.log("Eliminar producto:", id);
  }

  const products = [
    { id: 1, name: "Teclado" },
    { id: 2, name: "Ratón" }
  ];

  return (
    <div>
      {products.map(product => (
        <Product
          key={product.id}
          product={product}
          onDelete={handleDelete}
        />
      ))}
    </div>
  );
}
```

```
function Product({ product, onDelete }) {
  return (
    <article>
      <h2>{product.name}</h2>

      <button onClick={() => onDelete(product.id)}>
        Eliminar
      </button>
    </article>
  );
}
```

ibertech

end2end dedication

Tema 5

Formularios

Formularios controlados

- Un formulario es controlado cuando React es quien mantiene el valor actual de cada campo.
- En React, los formularios controlados siguen basándose en la misma idea fundamental: React mantiene el valor de los campos en el estado y el input refleja ese estado.

```
import { useState } from "react";

function Formulario() {
  const [nombre, setNombre] = useState("");

  return (
    <form>
      <input
        type="text"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />

      <p>Hola, {nombre}</p>
    </form>
  );
}
```

Inputs y value

- El valor de este input viene de la variable nombre.

```
const [nombre, setNombre] = useState("Carlos");

<input value={nombre} />
```

- Si posteriormente hacemos:

```
setNombre("Ana");
```

- React vuelve a renderizar y por eso se llama **controlled input**.
- Value es quién controla el input

onChange

- onChange nos permite reaccionar cuando cambia el input:

```
<input
  value={nombre}
  onChange={(e) => {
    setNombre(e.target.value);
  }}
/>
```

Estado del formulario

- Si el formulario tiene 3 campos se necesitan tres variables para guardar el estado.
- En formularios pequeños, seria viable
- Pero en formularios con muchos campos no lo es. En estos casos podemos utilizar otra técnica: Guardar el estado como objeto

```
function Registro() {
  const [nombre, setNombre] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  return (
    <form>
      <input
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
        placeholder="Nombre"
      />

      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />

      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Contraseña"
      />
    </form>
  );
```

Estado del formulario como objeto

- Podemos guardar todo el formulario en un único objeto:

```
const [formulario, setFormulario] = useState({
  nombre: "",
  email: "",
  password: "",
});
```

- Ahora tenemos:

```
formulario.nombre
formulario.email
formulario.password
```

```
<input
  value={formulario.nombre}
  onChange={(e) =>
    setFormulario({
      ...formulario,
      nombre: e.target.value,
    })
  }
/>
```

```
<input
  value={formulario.email}
  onChange={(e) =>
    setFormulario({
      ...formulario,
      email: e.target.value,
    })
  }
/>
```

Estado del formulario como objeto

- Un único onChange para todos los inputs:
- También es una buena práctica cuando el nuevo estado depende del estado anterior:

```
function handleChange(e) {
  const { name, value } = e.target;

  setFormulario((prev) => ({
    ...prev,
    [name]: value,
  }));
}
```

```
function Registro() {
  const [formulario, setFormulario] = useState({
    nombre: "",
    email: "",
    password: "",
  });

  function handleChange(e) {
    const { name, value } = e.target;

    setFormulario({
      ...formulario,
      [name]: value,
    });
  }

  return (
    <form>
      <input
        name="nombre"
        value={formulario.nombre}
        onChange={handleChange}
      />

      <input
        name="email"
        value={formulario.email}
        onChange={handleChange}
      />

      <input
        name="password"
        type="password"
        value={formulario.password}
        onChange={handleChange}
      />
    </form>
  );
```

Validación

- Para validar formularios en React hay separar tres cosas:

1. Los valores del formulario.

```
const [formulario, setFormulario] = useState({
  nombre: "",
  email: "",
  genero: "",
});

const [errores, setErrores] = useState({});
```

2. Los errores de validación.

```
{
  nombre: "El nombre es obligatorio",
  email: "El email no es válido",
  genero: "Selecciona una opción"
}
```

3. El momento en el que queremos mostrar esos errores.

-

Validación

3. El momento en el que queremos mostrar esos errores a través de la función de validación que se ejecuta al enviar el formulario.

```
function validarFormulario() {
  const nuevosErrores = {};

  if (!formulario.nombre.trim()) {
    nuevosErrores.nombre = "El nombre es obligatorio";
  }

  if (!formulario.email.trim()) {
    nuevosErrores.email = "El email es obligatorio";
  } else if (!formulario.email.includes("@")) {
    nuevosErrores.email = "Introduce un email válido";
  }

  if (!formulario.genero) {
    nuevosErrores.genero = "Selecciona una opción";
  }

  return nuevosErrores;
}
```

```
function handleSubmit(e) {
  e.preventDefault();

  const nuevosErrores = validarFormulario();

  setErrores(nuevosErrores);

  if (Object.keys(nuevosErrores).length > 0) {
    return;
  }

  console.log("Formulario válido");
}
```

Mensajes de error debajo del input

```
<div>
  <label>
    Nombre
  </label>

  <input
    name="nombre"
    value={formulario.nombre}
    onChange={handleChange}
  />

  {errores.nombre && (
    <p>{errores.nombre}</p>
  )}
</div>
```

checkbox

```
const [formulario, setFormulario] = useState({
  nombre: "",
  email: "",
  aceptaTerminos: false,
});
```

- Aquí no utilizamos value.

```
<input
  type="checkbox"
  name="aceptaTerminos"
  checked={formulario.aceptaTerminos}
  onChange={(e) => {
    setFormulario((prev) => ({
      ...prev,
      aceptaTerminos: e.target.checked,
    }));
  }}
/>
```

select

```jsx
const [pais, setPais] = useState("");

<select
  value={pais}
  onChange={(e) => setPais(e.target.value)}
>
  <option value="">Selecciona un país</option>
  <option value="es">España</option>
  <option value="fr">Francia</option>
  <option value="pt">Portugal</option>
</select>
```

radio

- Un **radio button** en un formulario controlado funciona de forma parecida al checkbox, pero hay una diferencia importante: para los radios normalmente controlamos el valor mediante **value**, no mediante checked de forma independiente.

```jsx
const [genero, setGenero] = useState("");

return (
  <form>
    <label>
      <input
        type="radio"
        name="genero"
        value="hombre"
        checked={genero === "hombre"}
        onChange={(e) => setGenero(e.target.value)}
      />
      Hombre
    </label>

    <label>
      <input
        type="radio"
        name="genero"
        value="mujer"
        checked={genero === "mujer"}
        onChange={(e) => setGenero(e.target.value)}
      />
      Mujer
    </label>
```

ibertech

end2end dedication

Tema 6

ReactJS y datos

useEffect

- useEffect permite ejecutar código que tiene efectos secundarios relacionados con el componente.
- Por ejemplo:
  - hacer una petición HTTP;
  - suscribirse a un evento;
  - iniciar un temporizador;
  - modificar algo externo a React;
  - conectarse a un WebSocket;
  - interactuar con APIs del navegador.

```
import { useEffect } from "react";

useEffect(() => {
  console.log("El componente se ha renderizado");
}, []);
```

[] es el array de dependencias

Ciclo de vida

- En React antiguo, se utilizaban métodos para gestionar el ciclo de vida:

```
componentDidMount
componentDidUpdate
componentWillUnmount
```

- Ahora, con versiones modernas useEffect permite gestionar muchos de esos casos:



useEffect sin dependencias

- Ejemplo:

```
useEffect(() => {
  console.log("Se ejecuta después de cada render");
});
```

- Cada vez que el componente renderiza:

```
render
 ↓
effect

render
 ↓
effect

render
 ↓
effect
```

useEffect con array vacio

- Es un patrón muy habitual para cargar datos inicialmente.

```
useEffect(() => {
  fetch("/api/usuarios");
}, []);
```

```
montaje
   ↓
render
   ↓
useEffect
```

useEffect con dependencias

- Tenemos un usuario y queremos cargar sus datos:

```
const [usuarioId, setUsuarioId] = useState(1);
```

```
useEffect(() => {
  fetch(`/api/usuarios/${usuarioId}`);
}, [usuarioId]);
```

```
usuarioId = 1
      ↓
fetch /usuarios/1

usuarioId = 2
      ↓
fetch /usuarios/2

usuarioId = 3
      ↓
fetch /usuarios/3
```

Peticiones HTTP con fetch

- JavaScript proporciona la API fetch.

```
const response = await fetch(
  "https://api.example.com/usuarios"
);
```

- fetch() no considera automáticamente un HTTP 404 o 500 como una excepción, por eso hacemos lo siguiente:

```
if (!response.ok) {
  throw new Error("Error HTTP");
}
```

- Después podemos obtener JSON:

```
const datos = await response.json();
```

Estado de carga

```
const [loading, setLoading] = useState(true);
```

```
if (loading) {
  return <p>Cargando...</p>;
}
```

Estado de error

```
const [error, setError] = useState(null);
```

```
setError(error.message);
```

```
if (error) {
  return <p>Error: {error}</p>;
}
```

ibertech

end2end dedication

Tema 7

ReactJS y servidor

API REST

- Una **API REST** es una forma habitual de que el frontend y el backend se comuniquen mediante HTTP.
- Métodos principales:

Método	Uso habitual
`GET`	Obtener datos
`POST`	Crear datos
`PUT`	Actualizar un recurso
`DELETE`	Eliminar un recurso

GET

Forma tradicional:

```
const response = await fetch("/api/usuarios");

if (!response.ok) {
  throw new Error("Error al obtener usuarios");
}

const usuarios = await response.json();
```

Con useEffect:

```
useEffect(() => {
  async function cargarUsuarios() {
    try {
      setLoading(true);
      setError(null);

      const response = await fetch("/api/usuarios");

      if (!response.ok) {
        throw new Error("No se pudieron cargar los usuarios");
      }

      const datos = await response.json();

      setUsuarios(datos);
    } catch (error) {
      setError(error.message);
    } finally {
      setLoading(false);
    }
  }
```

POST

```
const response = await fetch("/api/usuarios", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    nombre: "Laura",
    email: "laura@email.com",
  }),
});
```

PUT

```
const response = await fetch("/api/usuarios/15", {
  method: "PUT",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    nombre: "Ana García",
    email: "ana.garcia@email.com",
  }),
});
```

DELETE

```
async function eliminarUsuario(id) {
  const response = await fetch(
    `/api/usuarios/${id}`,
    {
      method: "DELETE",
    }
  );

  if (!response.ok) {
    throw new Error("No se pudo eliminar");
  }
}
```

ibertech

end2end dedication

Muchas gracias por vuestra asistencia

www.ibertech.es